

به نام خدا

گزارش دوم آزمایشگاه
سیستم عامل

سید علیرضا میرشفیعی - ۸۱۰۱۰۱۵۳۲

محمد تقی زاده - ۸۱۰۱۰۲۵۸۹

محمد صدرا عباسی - ۸۱۰۱۰۱۴۶۹

(۱)

چگونگی پنهان سازی جزئیات فراخوانی های سیستمی:

با توجه به تعریف ماکروی SYSCALL در فایل `usys.s`، جزئیات سطح پایین مربوط به پیاده سازی یک فراخوانی سیستمی، از جمله:

- قرار دادن شماره‌ی `system call` مربوطه در رجیستر `eax`
- پرتاب وقفه نرم افزاری (`int $T_SYSCALL`) به هسته برای انتقال کنترل
- و دریافت مقدار بازگشتی از هسته و ادامه اجرای برنامه سطح کاربر پس از بازگشت

همگی از دید برنامه نویسی پنهان می شوند. بنابراین، برنامه نویسی صرفاً با فرض انجام صحیح این مراحل در پس زمینه، می تواند بر پیاده سازی منطق `system call` در فایل هایی مانند `sysproc.c` و `proc.c` تمرکز کند، بدون نیاز به درگیر شدن با جزئیات انتقال کنترل بین `user space` و `kernel space`. همچنین پیاده سازی توابع پایه ای که در بسیاری از برنامه های سطح کاربر کاربرد دارند در فایل `ulib.c` آمده است.

دلایل استفاده از abstraction:

- از اونجایی که جزئیات پیاده سازی سطح پایین از دید برنامه نویسی پنهان است قابلیت توسعه آن افزایش میابد چون بیشتر درگیر پیاده سازی منطق اصلی برنامه می شود
- به دلیل عدم دسترسی مستقیم برنامه نویسی به رجیستر ها احتمال باگ کمتر می شود
- اگر بخواهیم در معماری سیستم عامل تغییر اساسی ایجاد کنیم لازم است تنها فایل های سطح پایین رو تغییر دهیم و منطق اصلی برنامه های سطح کاربر تغییر نمی کند

(۲)

۱. نحوه پیاده سازی و عملکرد

- دستور `int` (رویکرد `xv6`):

این یک دستورالعمل عمومی برای ایجاد «وقفه نرم افزاری» (`Software Interrupt`) در `x86` است. در `xv6`، برنامه سطح کاربر دستور `int 64` را اجرا می کند. پردازنده با مراجعه به «جدول توصیفگر وقفه» (`IDT`)، گیت (`Gate`) شماره ۶۴ را پیدا می کند که به عنوان یک `Trap Gate` با سطح دسترسی مجاز (`DPL_USER`) تنظیم شده است. پس از آن، پردازنده به صورت سخت افزاری و خودکار، وضعیت پردازنده (شامل `CS`، `eip`، `eflags`، `esp`، `SS`) را روی پشته هسته (`Kernel Stack`) ذخیره (`Push`) می کند و به

آدرس «کنترل‌کننده تله» (Trap Handler) می‌پرد. در این روش، آرگومان‌ها معمولاً توسط توابع پوشاننده (wrapper) روی پشته سطح کاربر (User Stack) قرار می‌گیرند.

- **دستور sysenter (رویکرد لینوکس):**

این دستور (و دستور مشابه sysexit) به طور خاص برای اجرای سریع فراخوانی‌های سیستمی طراحی شده است. این روش از IDT استفاده نمی‌کند. در عوض، هسته (مانند لینوکس) هنگام بوت، آدرس روتین ورودی خود و آدرس پشته هسته را در ثبات‌های خاص CPU (به نام MSRs) ثبت می‌کند. اجرای sysenter باعث می‌شود پردازنده مستقیماً به آن آدرس پریده و به مد هسته سوییچ کند که بسیار سریع‌تر از جستجو در جداول و بررسی‌های امنیتی int است. برای افزایش سرعت، لینوکس آرگومان‌ها را به جای پشته، مستقیماً از طریق ثبات‌ها (Registers) منتقل می‌کند.

۲. کارایی (Performance)

- دستور int: این روش دارای «سربار مستقیم» (Direct Overhead) بالایی است. مراحل سخت‌افزاری متعدد آن (مانند بررسی‌های امنیتی، دسترسی به حافظه برای IDT، و Push کردن کامل وضعیت روی پشته) زمان‌بر هستند و باعث کاهش کارایی می‌شوند.
- دستور sysenter: این دستور برای به حداقل رساندن «سربار انتقال» (Transfer Overhead) طراحی شده است. به دلیل حذف مراحل میانی و پرش مستقیم، سریع‌ترین مکانیزم موجود برای ورود و خروج به هسته در پردازنده‌های x86 است.

۳. کاربردهای مختلف

- اولویت سادگی (xv6): دستور int با وجود کندی، پیاده‌سازی ساده‌تری دارد. برای یک سیستم عامل آموزشی مانند xv6 که شفافیت و سادگی کد بر کارایی محض اولویت دارد، انتخاب مناسبی است.
- اولویت کارایی (Linux/Windows): در سیستم‌عامل‌های عملیاتی که حجم عظیمی از فراخوانی‌های سیستمی را پردازش می‌کنند، استفاده از sysenter برای حفظ پرفورمنس سیستم ضروری است.
- مورد خاص (vDSO): برای برخی فراخوانی‌های بسیار سبک و پرتکرار (مانند gettimeofday در لینوکس)، حتی سربار sysenter نیز قابل توجه است. در این موارد، لینوکس از مکانیزم vDSO استفاده می‌کند که پیاده‌سازی تابع را مستقیماً در فضای آدرس کاربر نگاشت (map) می‌کند تا فراخوانی بدون هیچ‌گونه سوییچ به مد هسته انجام شود.

تنها تله‌ای که به صورت مستقیم توسط برنامه‌های فضای کاربر فعال می‌شود، system call است. در این حالت، برنامه‌ی کاربر با فعال‌سازی یک Gate از نوع Trap که دارای سطح دسترسی $DPL_USER = 3$ است، درخواست خود را به کرنل ارسال می‌کند اما وقفه‌های سخت‌افزاری و استثنائات (Exceptions) توسط سخت‌افزار یا پردازنده به طور خودکار ایجاد می‌شوند و نباید توسط فضای کاربر صدا زده شوند. به همین دلیل، Gate های مربوط به این تله‌ها دارای $DPL = 0$ هستند تا فقط از سمت کرنل یا CPU قابل فعال‌سازی باشند. در غیر این صورت امنیت و ثبات کرنل به خطر می افتد.

(۴)

این کار به این دلیل انجام می‌شود که در صورت تغییر privilege level، لازم است پردازنده از یک پشته‌ی جدید (مختص سطح جدید) استفاده کند و برای بازگشت صحیح، آدرس پشته‌ی قبلی (ss و esp) نیز حفظ شود. اما اگر سطح دسترسی تغییر نکند، نیازی به تغییر پشته نیست و بنابراین push این مقادیر ضرورتی ندارد.

(۵)

`argint(int n, int *ip)` : از این تابع برای گرفتن آرگومان های integer یک system call استفاده می شود. در تابع مشخص شده n نشان دهنده شماره پارامتری است که باید دریافت شود. این تابع برای دریافت آرگومان ها از user stack از رجیستری به نام esp استفاده میکند. این رجیستر به بالاترین نقطه استک اشاره می کند. نحوه بدست آوردن مکان دقیق پارامتر مورد نظر نیز به صورت $esp+4+4*n$ است.

`argptr(int n, char **pp, int size)` : از این تابع برای گرفتن n امین آرگومان argument که systemcall است استفاده می شود.

argstr(int n, char **pp) : از این تابع برای گرفتن آرمین آرگومان systemcall که اشاره گری به یک string است استفاده می شود. وظیفه دیگر تابع این است که string موردنظر valid است و به null ختم می شود. دلیلی که argptr بازه های آدرس ورودی را بررسی می کند این است که اشاره گر مورد نظر در محدوده اختصاص یافته به process حضور داشته باشد. از طرف دیگر بررسی می کند که اشاره گر به بخش معتبری از حافظه با اندازه مشخص دسترسی پیدا کند. این مرحله اطمینان حاصل می کند که user process به kernel mory دسترسی پیدا نکند.

```
int
sys_read(void)
{
    struct file *f;
    int n;
    char *p;

    if(argfd(n: 0, pfd: 0, pf: &f) < 0 || argint(2, &n) < 0 || argptr(1, &p, n) < 0)
        return -1;
    return fileread(f, p, n);
}
```

در sys_read در صورتی که argptr بازه آدرس ورودی را بررسی نمی کرد، user program مقدار p را می توانست اشاره گر به یک آدرس غیرقابل قبول یا اشاره گر به kernel memory انتخاب کند. در عمل می توان تابعی پیاده سازی کرد که بازه ورودی را چک کند و صرفاً پوینتر را از استک پردازنده استخراج و دریافت کند اما در این صورت user program به سادگی می تواند وارد فضای کرنل شود و امنیت سیستم را به خطر بیندازد به طور مثال می تواند آدرسی را در فضا کرنل به سیستم کال بدهد و اگر سیستم کال داده ای را درون آن کپی کند عملاً برنامه user در کرنل داده overwrite می کند

(6)

اگر کاربر به صورت دستی رجیسترهایی مانند `edi`, `esi`, `ebx` و `ebp` را تغییر دهد، دو مشکل اساسی رخ می‌دهد. اولاً، چون لینوکس پارامترها را از طریق رجیسترها منتقل می‌کند، این کار باعث ارسال آرگومان‌های اشتباه به هسته می‌شود. ثانیاً، و مهم‌تر، این کار «واسط باینری برنامه» (ABI) را نقض می‌کند. کامپایلر فضای کاربر این رجیسترها را `callee-saved` فرض می‌کند، یعنی انتظار دارد مقادیرشان پس از بازگشت از فراخوانی سیستمی دست‌نخورده باقی بماند. اما هسته هیچ تعهدی به این قرارداد ندارد و آزادانه از این رجیسترها برای محاسبات داخلی خود استفاده می‌کند. در نتیجه، وقتی کنترل به برنامه برمی‌گردد، مقادیر حیاتی (مانند اشاره‌گر پایه پشته در `ebp`) با داده‌های نامربوط هسته رونویسی شده‌اند که این وضعیت به سرعت منجر به خطای Segmentation Fault و از کار افتادن برنامه می‌شود. دقیقاً به همین دلیل، توابع پوشاننده (wrapper) در `glibc` وجود دارند تا این رجیسترها را قبل از ورود به هسته ذخیره (push) و پس از بازگشت، بازیابی (pop) کنند و این قرارداد را حفظ نمایند.

بررسی گام های اجرای فراخوانی سیستمی در سطح کرنل توسط gdb

ابتدا یک برنامه کاربر ساده می نویسیم که سیستم کال انجام دهد:

```
#include "types.h"
#include "user.h"

Windsurf: Refactor | Explain | Generate Function Comment | ×
int main(int argc, char *argv[]) {
    int pid = getpid();

    printf(1, "PID result: (%d)\n", pid);
    exit();
}
```

سپس سیستم عامل را در حالت دیباگ اجرا کرده و gdb را به آن متصل می کنیم. برای این کار، ابتدا این دستور را اجرا می کنیم: `make qemu-gdb`

سپس در یک ترمینال جدا، gdb را اجرا کرده و به پورت دیباگ متصل می شویم:

`gdb kernel`

`(gdb) target remote:26000`

```
sadra@sadra-ThinkPad-X1-Yoga-4th:~/Documents/Codes/xv6-public$ make qemu-gdb
*** Now run 'gdb'.
qemu-system-i386 -serial mon:stdio -drive file=fs.img,index=1,media=disk,format=raw -drive file=xv6.img,index=0,media=disk,format=raw -smp 2 -m 512 -S -gdb tcp
::26000
(gdb) target remote :26000
Remote debugging using :26000
0x00000000 in ?? ()
(gdb)
```

سپس روی syscall یک brake point ست می کنیم

```
For more information about this security protection see the
"Auto-loading safe path" section in the GDB manual.  E.g., run from the shell:
  info "(gdb)Auto-loading safe path"
(gdb) break syscall
Breakpoint 1 at 0x80105820: file syscall.c, line 181.
(gdb) █
```

اجرا را تا رسیدن به نقطه مورد نظر ادامه می دهیم سپس برنامه کاربر را در کنسول فراخوانی می کنیم و با یک bt خروجی به صورت زیر خواهد بود:

```
(gdb) bt
#0  syscall () at syscall.c:181
#1  0x801069ed in trap (tf=0x8dffefb4) at trap.c:43
#2  0x8010678c in alltraps () at trapasm.S:20
#3  0x8dffefb4 in ?? ()
Backtrace stopped: previous frame inner to this frame (corrupt stack?)
```

- **دستور bt** در GDB برای مشاهده‌ی پشت‌های فراخوانی (call stack) استفاده می‌شود. با اجرای این دستور، می‌توان توالی توابعی را که منجر به وضعیت فعلی اجرای برنامه شده‌اند را مشاهده کرد.
- **تحلیل خروجی** نشان می‌دهد که فراخوانی ابتدا از فضای کاربر آغاز شده، سپس وارد مسیر alltraps در کد assembly شده و از آنجا به تابع trap در کرنل منتقل شده است. در trap، پس از ساخت trapframe و تشخیص شماره تله برابر با T_SYSCALL، کنترل به تابع syscall منتقل می‌شود. در syscall، با استفاده از شماره system call (مقدار eax) متناظر از آرایه syscalls فراخوانی می‌شود. این خروجی نحوه‌ی ورود از فضای کاربر به هسته و مسیر اجرای سیستم کال را نشان می‌دهد
- **کاربرد دستور down و up:** دستور down در GDB برای حرکت به فریم پایین‌تر در پشت‌های فراخوانی استفاده می‌شود؛ یعنی به تابعی که تابع فعلی را فراخوانی کرده است. این دستور برای بررسی context توابع قبلی، آرگومان‌های ورودی و محل دقیق فراخوانی آن‌ها کاربرد دارد. دستور up در مقابل، برای بازگشت به فریم بالاتر در stack به کار می‌رود؛ یعنی به تابعی که توسط فریم فعلی فراخوانی شده است. این ابزار کمک می‌کند در پشت‌های جابه‌جا شویم و وضعیت اجرای برنامه در سطوح مختلف را تحلیل کنیم.

در ادامه باید از دستور down استفاده کرده و محتوای رجیستر eax را تحلیل کنیم:

```
(gdb) down
Bottom (innermost) frame selected; you cannot go down.
(gdb) █
```

این پیغام نشان می دهد که در پایین ترین سطح پشته قرار داریم و امکان رفتن به فریم پایین تر وجود ندارد. بنابراین با اجرای دستور up یک سطح به بالا برمی گردیم و وارد فریم مربوط به تابع trap می شویم. در این فریم، به ساختار trapframe دسترسی داشته و با استفاده از دستور p tf->eax مقدار شماره system call را استخراج می کنیم:

```
(gdb) up
#1 0x801069ed in trap (tf=0x8dffefb4) at trap.c:43
43      syscall();
(gdb) p tf->eax
$1 = 5
(gdb) █
```

مقدار شماره سیستم کال برابر ۵ است و این به این دلیل است که قبل از اجرای اصلی برنامه کاربر ، هسته ی سیستم عامل ابتدا نیاز دارد ورودی کاربر را پردازش کند. این ورودی خوانده می شود و برای این کار، system call شماره ۵ که مربوط به read است، فراخوانی می شود. پس از آن، سیستم کال های دیگری مانند fork، exec، و در نهایت getpid اجرا خواهند شد. بنابراین مشاهده ی system call read پیش از getpid در مسیر اجرای پشته (call stack) نشان می دهد که هنوز به نقطه ای نرسیده ایم که getpid فراخوانی شده باشد.

اجرا را ادامه می دهیم تا شماره سیستم کال برابر getpid یا همان ۱۱ شود:

```
#1 0x801069ed in trap (tf=0x8dffe4b4) at trap.c:43
43      syscall();
(gdb) p tf->eax
$6 = 5
(gdb) c
Continuing.

Thread 1 hit Breakpoint 1, syscall () at syscall.c:181
181      struct proc *curproc = myproc();
(gdb) up
#1 0x801069ed in trap (tf=0x8dffe4b4) at trap.c:43
43      syscall();
(gdb) p tf->eax
$7 = 1
(gdb) c
Continuing.

Thread 1 hit Breakpoint 1, syscall () at syscall.c:181
181      struct proc *curproc = myproc();
(gdb) up
#1 0x801069ed in trap (tf=0x8dffe4b4) at trap.c:43
43      syscall();
(gdb) p tf->eax
$8 = 3
(gdb) c
Continuing.

Thread 1 hit Breakpoint 1, syscall () at syscall.c:181
181      struct proc *curproc = myproc();
(gdb) up
#1 0x801069ed in trap (tf=0x8df1bfb4) at trap.c:43
43      syscall();
(gdb) p tf->eax
$9 = 12
(gdb) c
Continuing.

Thread 1 hit Breakpoint 1, syscall () at syscall.c:181
181      struct proc *curproc = myproc();
(gdb) up
#1 0x801069ed in trap (tf=0x8df1bfb4) at trap.c:43
43      syscall();
(gdb) p tf->eax
$10 = 7
(gdb) c
Continuing.

Thread 1 hit Breakpoint 1, syscall () at syscall.c:181
181      struct proc *curproc = myproc();
(gdb) up
#1 0x801069ed in trap (tf=0x8df1bfb4) at trap.c:43
43      syscall();
(gdb) p tf->eax
$11 = 11
(gdb)
```

ارسال آرگومان های فراخوانی سیستمی

۱. سفارشی سازی تابع پوشاننده (Wrapper) در سطح کاربر

ابتدا باید تابع پوشاننده (wrapper) سطح کاربر را برای فراخوانی سیستمی جدید خود سفارشی کنیم.

```
#define SYSCALL(name) \
    .globl name; \
    name: \
        movl $SYS_ ## name, %eax; \
        int $T_SYSCALL; \
        ret
```

در حالت پیش فرض، xv6 از ماکروی SYSCALL (در فایل usys.S) برای تولید توابع پوشا استفاده می کند. این ماکروی استاندارد، ابتدا شماره سیستم کال مربوطه را در رجیستر EAX قرار می دهد و سپس دستور وقفه نرم افزاری (int \$T_SYSCALL) را اجرا می کند.

با اجرای دستور int:

۱. CPU به صورت سخت افزاری اجرای کد کاربر را متوقف کرده و به حالت هسته (Kernel Mode) می رود.

۲. با مراجعه به جدول توصیفگر وقفه (IDT)، آدرس هندلر مربوطه (در اینجا vector64) را پیدا می کند.

۳. هسته، پس از ذخیره سازی کامل وضعیت پردازش (از جمله رجیسترها) در قاب تله (Trap Frame)، در نهایت تابع trap() را برای رسیدگی به وقفه اجرا می کند.

پس از اتمام کار در هسته و بازگشت به فضای کاربر، دستور ret (که در انتهای ماکروی SYSCALL قرار دارد) اجرا شده، آدرس بازگشت را از پشته کاربر برداشته و به ادامه اجرای برنامه jump می کند.

در ادامه ما باید تابع پوشای سفارشی خود را به گونه‌ای در `usys.S` تعریف کنیم که دو پارامتر ورودی را قبل از اجرای وقفه، در رجیسترهای `EBX` و `ECX` قرار دهد. این آرگومان‌ها قبل از `call` کردن تابع، روی پشته قرار گرفته‌اند. وظیفه ما خواندن آن‌ها از پشته و انتقالشان به رجیسترهای مورد نظر است

```
.globl simple_arithmetic_syscall
simple_arithmetic_syscall:
    movl 4(%esp), %ebx // a parameter to ebx register
    movl 8(%esp), %ecx // b parameter to ecx register

    movl $SYS_simple_arithmetic_syscall, %eax // syscall number to eax register
    int $T_SYSCALL
    ret
```

۲. اضافه کردن هدر تابع به `user.h` تا برنامه‌های سطح کاربر بتوانند آن را بشناسند

```
int dup(int);
int getpid(void);
char* sbrk(int);
int sleep(int);
int uptime(void);
int simple_arithmetic_syscall(int, int);
```

۳. اضافه کردن شماره سیستم کال به `syscall.h`

در این مرحله، باید یک شماره یکتا به `SYS_simple_arithmetic_syscall` در فایل `syscall.h` اختصاص دهیم. این شماره، همان مقداری است که در تابع پوشاننده (`usys.S`) داخل رجیستر `EAX` قرار دادیم و به هسته اجازه می‌دهد تا این فراخوانی سیستمی را شناسایی کند.

```
#define SYS_write 16
#define SYS_mknod 17
#define SYS_unlink 18
#define SYS_link 19
#define SYS_mkdir 20
#define SYS_close 21
#define SYS_simple_arithmetic_syscall 22
```

۴. ثبت در جدول سیستم کال ها در فایل syscall.c

سیستم کال خود را به lookup table پوینتر به فانکشن سیستم کال ها اضافه می کنیم تا وقتی trap اجرا شد با دریافت شماره سیستم کال، تابع مربوطه در کرنل را فراخوانی و اجرا کند

```
static int (*syscalls[])(void) = {
[SYS_fork]      sys_fork,
[SYS_exit]      sys_exit,
[SYS_wait]      sys_wait,
[SYS_pipe]      sys_pipe,
[SYS_read]      sys_read,
[SYS_kill]      sys_kill,
[SYS_exec]      sys_exec,
[SYS_fstat]     sys_fstat,
[SYS_chdir]     sys_chdir,
[SYS_dup]       sys_dup,
[SYS_getpid]    sys_getpid,
[SYS_sbrk]      sys_sbrk,
[SYS_sleep]     sys_sleep,
[SYS_uptime]    sys_uptime,
[SYS_open]      sys_open,
[SYS_write]     sys_write,
[SYS_mknod]     sys_mknod,
[SYS_unlink]    sys_unlink,
[SYS_link]      sys_link,
[SYS_mkdir]     sys_mkdir,
[SYS_close]     sys_close,
[SYS_simple_arithmetic_syscall] sys_simple_arithmetic_syscall
};
```

۵. پیاده سازی بدنه تابع در کرنل و در فایل sysproc.c

```
int
sys_simple_arithmetic_syscall(void)
{
    int a, b, result;
    struct proc *curproc = myproc();

    a = curproc->tf->ebx; // read 'a' from EBX in the current process kernel stack
    b = curproc->tf->ecx; // read 'b' from ECX in the current process kernel stack

    result = (a + b) * (a - b); //

    cprintf("Calc: (%d+%d)*(%d-%d)=%d\n", a, b, a, b, result);

    return result;
}
```

پارامترها به طور مستقیم از محتوای رجیسترهای قرار گرفته در استک کرنل خوانده می شوند

۶. نوشتن یک برنامه کاربر برای تست و اجرای سیستم کال

```
int
main(int argc, char *argv[])
{
    int a = 5;
    int b = 3;
    int result = simple_arithmetic_syscall(a, b);
    printf(1, "Return value: %d\n", result);
    exit();
}
```

۷. نتیجه و خروجی

```
$ arithmetic_test
Calc: (5+3)*(5-3)=16
Return value: 16
$ _
```